matplotlib
笔记本

吴嘉军

2026 年 1 月 11 日

摘要

这里是摘要

关键词： 这里是关键词 1、关键词 2

目录

§1、 简单入门 **1**
1、 安装 1
1.1、 导入库 1
1.2、 创建数据 1
2、 绘制图形 1
3、 插入中文 1
4、 坐标布局 2
5、 坐标轴 2
6、 箭头 3
7、 文字说明 4

§2、 patches **5**
1、 矩形 5

Wu Jia Jun

§1、 简单入门

一般来说,matplotlib 需要和 numpy, 搭配着使用, 因为 matplotlib 主要是用来绘图的, 而 numpy 主要是用来进行数值计算的, 两者结合起来可以更好地进行数据处理和可视化.

1、 安装

在使用 matplotlib 之前, 需要先安装它. 可以使用 pip 命令进行安装

```
pip install matplotlib
```

安装完成后, 可以使用以下命令来验证是否安装成功.

1.1、 导入库

在使用 matplotlib 之前, 需要先导入它. 通常情况下, 我们会导入 matplotlib.pyplot 模块, 并将其重命名为 plt. 这样可以方便我们后续的使用.

```
import matplotlib.pyplot as plt
import numpy as np
```

1.2、 创建数据

在绘图之前, 我们需要先创建一些数据. 通常情况下, 我们会使用 numpy 来创建数据. 例如, 我们可以使用 numpy 的 linspace 函数来创建一个等间隔的数组.

```
x = np.linspace(0, 10, 100) # 创建一个从0到10的等间隔数组,包含100个点
y = np.sin(x) # 计算x对应的正弦值
```

2、 绘制图形

使用 matplotlib 绘制图形非常简单. 我们只需要调用 plt.plot 函数, 并传入 x 和 y 数据即可. 然后, 使用 plt.show 函数来显示图形.

```
plt.plot(x, y) # 绘制图形
plt.xlabel('X轴标签') # 设置X轴标签
plt.ylabel('Y轴标签') # 设置Y轴标签
plt.title('图形标题') # 设置图形标题
plt.show() # 显示图形
```

下面是一个简单的例子, 展示如何使用 matplotlib 绘制一个正弦波.

```
import matplotlib.pyplot as plt
import numpy as np
# 创建数据
x = np.linspace(0, 4 * np.pi, 100)
y = np.sin(x)

# 绘制图形
plt.plot(x, y)
plt.xlabel('X')
plt.ylabel('Y')
plt.title('sin')

plt.savefig('figures/section1/1.1.png') # 保存图形为文件
# Show the plot
plt.show()
```

运行上述代码, 将会显示一个包含正弦波的图形窗口.

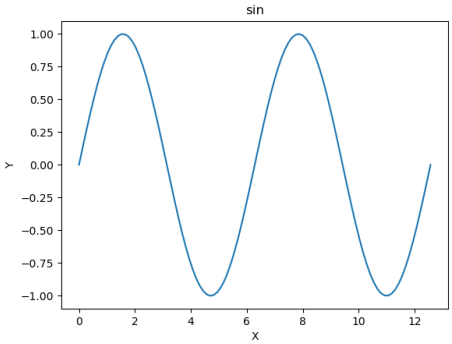

图 1.1: 正弦波图形

3、 插入中文

在使用 matplotlib 绘图时, 如果需要插入中文, 可以使用以下方法:

```
plt.xlabel('X轴标签', fontproperties='SimHei') # 设置X轴标签为中文
plt.ylabel('Y轴标签', fontproperties='SimHei') # 设置Y轴标签为中文
plt.title('图形标题', fontproperties='SimHei') # 设置图形标题为中文
```

这里的'SimHei' 是黑体字体的名称, 你也可以根据需要选择其他字体. 确保你的系统中已经安装了所需的中文字体.

```
import matplotlib.pyplot as plt
import numpy as np
# 创建数据
x = np.linspace(0, 4 * np.pi, 100)
y = np.sin(x)

# 绘制图形
plt.plot(x, y)
plt.xlabel('X轴标签', fontproperties='SimHei') # 设置X轴标签为中文
plt.ylabel('Y轴标签', fontproperties='SimHei') # 设置Y轴标签为中文
plt.title('图形标题', fontproperties='SimHei') # 设置图形标题为中文

plt.savefig('figures/section1/1.2.png') # 保存图形为文件
# Show the plot
```

```
plt.show()
```

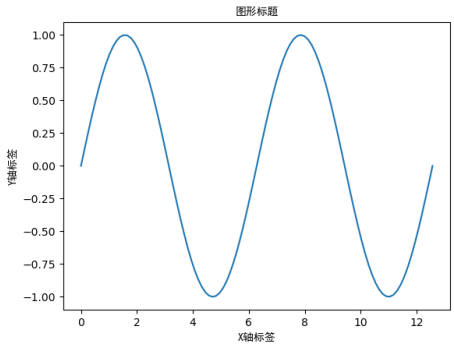

图 1.2

如果觉得太麻烦了, 可以在代码开头添加如下代码, 这样就可以全局使用中文字体了.

```
import matplotlib.pyplot as plt
from matplotlib import rcParams
rcParams['font.sans-serif'] = ['SimHei'] # 全局设置中文字体为黑体
rcParams['axes.unicode_minus'] = False # 解决负号显示问题
```

运行上述代码, 将可以在 fig 中添加中文, 以下代码为例:

```
import matplotlib.pyplot as plt
import numpy as np
# 创建数据
plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体为SimHei
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题
x = np.linspace(0, 4 * np.pi, 100)
y = np.sin(x)

# 绘制图形
plt.plot(x, y)
plt.xlabel('X轴标签') # 设置X轴标签为中文
plt.ylabel('Y轴标签') # 设置Y轴标签为中文
plt.title('图形标题') # 设置图形标题为中文

plt.savefig('figures/section1/1.3.png') # 保存图形为文件
# Show the plot
plt.show()
```

4、 坐标布局

在 matplotlib 中, 可以通过多种方式来调整图形的坐标布局. 以下是一些常用的方法:

```
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
fig = plt.figure("title") # an empty figure with no Axes
fig, ax = plt.subplots() # a figure with a single Axes
fig.suptitle("Figure title") # set the Figure title
ax.set_title("Axes title") # set the Axes title
fig, axs = plt.subplots(2, 2) # a figure with a 2x2 grid of Axes
axs[0, 0].set_title("Axes [0,0] title") # set the title for a specific Axes
axs[1, 1].set_title("Axes [1,1] title")
fig.tight_layout() # adjust spacing between subplots

# a figure with one Axes on the left, and two on the right:
fig, axs = plt.subplot_mosaic([['left', 'right_top'],
                               ['left', 'right_bottom']])
axs['left'].set_title("左侧子图标题")
axs['right_top'].set_title("右上子图标题")
axs['right_bottom'].set_title("右下子图标题")
fig.tight_layout() # adjust spacing between subplots
plt.savefig('figures/section1/1.4.png') # 保存图形为文件
plt.show()
```

运行上述代码, 将会显示一个包含三个子图的图形窗口, 一个在左, 两个在右. 如下所示:

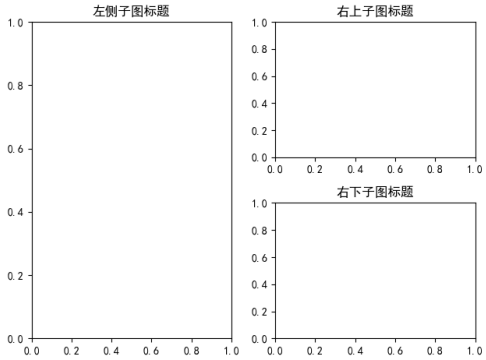

图 1.3: 多个子图布局

5、 坐标轴

在 matplotlib 中, 可以通过多种方式来调整图形的坐标轴. 以下是一些常用的方法:

```
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

fig,ax = plt.subplots()
x = np.arange(0, 4*np.pi, 0.01)
y = np.sin(x)
ax.plot(x, y)
# 设置坐标轴标签
ax.set_xlabel("X轴标签")
ax.set_ylabel("Y轴标签")
# 设置坐标轴范围
ax.set_xlim(0, 9)
ax.set_ylim(-1, 1)
# 设置刻度
ax.set_xticks([0, 0.5*np.pi, np.pi, 1.5*np.pi, 2*np.pi, 2.5*np.pi,3*np.pi,3.5*np.pi,4*np.pi])
ax.set_yticks([-1, -0.8, -0.6, -0.4, -0.2, 0, 0.2, 0.4, 0.6, 0.8, 1])

# 设置刻度标签
# ax.set_xticklabels(['0', '零点五 ', ' ', '一点五 ', '2 ', '二点五 ','3 ','三点五 ','4 '])
# ax.set_yticklabels(['-1', '-0.8', '-0.6', '-0.4', '-0.2', '0', '0.2 ', '0.4', '0.6', '0.8', '1'])
# 设置标题
ax.set_title("坐标轴示例")
```

```python
# 显示网格
ax.grid(True)
# 上下左右边框可见性
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
# 设置刻度方向、长度、宽度和颜色
ax.tick_params(direction='inout', length=10, width=10, colors='red')
# 设置刻度线位置
ax.xaxis.set_ticks_position('bottom')
ax.yaxis.set_ticks_position('left')
# 设置刻度线样式
# 打开次刻度线
# 关闭次刻度线
ax.minorticks_off()
# 显示主次网格线
ax.grid(which='minor', linestyle='--', linewidth=0.5)
ax.minorticks_on()
ax.xaxis.set_tick_params(which='major', size=10, width=1, direction='out',colors='blue')
ax.xaxis.set_tick_params(which='minor', size=5, width=1, direction='out', colors='red')
ax.yaxis.set_tick_params(which='major', size=10, width=0.5, direction='out')
ax.yaxis.set_tick_params(which='minor', size=5, width=1, direction='in', colors='green')

# 设置坐标轴0位置
ax.spines['bottom'].set_position(('data', 0)) # x轴位置
ax.spines['left'].set_position(('data', 0)) # y轴位置

# 添加箭头 ,clip_on=False的作用是防止箭头被裁剪掉
ax.plot(1, 0, '>r',transform=ax.get_yaxis_transform(),clip_on=False) # transform=ax.get_yaxis_transform()表示箭头在y轴上
ax.plot(0, 1, '^b', transform=ax.get_xaxis_transform(),clip_on=False) # transform=ax.get_xaxis_transform()表示箭头在x轴上

plt.savefig('figures/section1/1.5.png') # 保存图形为文件
plt.show()
```

运行上述代码, 将会显示一个包含自定义坐标轴的图形窗口. 如下所示:

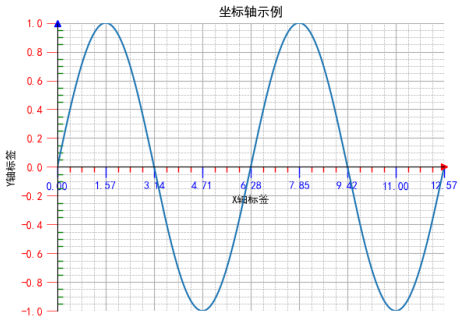

图 1.4: 自定义坐标轴

6、 箭头

在 matplotlib 中, 可以通过多种方式来添加箭头. 以下是一些常用的方法:

```python
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

fig, ax = plt.subplots()

# 弯箭头示例
arrow = mpatches.FancyArrowPatch(
    # 起点和终点坐标
    (0.2, 0.2), (.8, .8),
    # 箭头样式和属性
    arrowstyle='->,head_length=0.4,head_width=0.2',
    connectionstyle='arc3,rad=0.3', # 弯曲半径
    # 其他属性,箭头颜色,线宽,大小等
    color='blue', linewidth=2, mutation_scale=20,
    # 坐标系是data坐标系,不裁剪
    transform=ax.transData, clip_on=False
)
ax.add_patch(arrow)

plt.savefig('figures/section1/1.6.png') # 保存图形为文件
plt.show()
```

运行上述代码, 将会显示一个包含箭头的图形窗口. 如下所示:

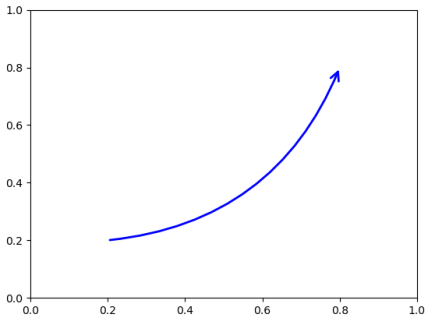

图 1.5: 箭头示例

再举例说明一下如何添加不同样式的箭头.

```python
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np

plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

fig, ax = plt.subplots()
x = np.linspace(0, 4*np.pi, 100)
y = np.sin(x)
ax.set_title("箭头示例2")
ax.plot(x, y)
# 直箭头示例
arrow1 = mpatches.FancyArrowPatch(
    (0.7*np.pi, 1.2), (0.5 * np.pi, 1),
    arrowstyle='->', mutation_scale=20,
    connectionstyle='arc3,rad=0.3',
    color='red',
    transform=ax.transData, clip_on=False
)
arrow2 = mpatches.FancyArrowPatch(
    (3.7*np.pi, -1.2), (3.5 * np.pi, -1),
    arrowstyle='->', mutation_scale=20,
    connectionstyle='arc3,rad=0.3',
    color='red',
    transform=ax.transData, clip_on=False
```

```python
)

ax.add_patch(arrow1)
ax.add_patch(arrow2)

ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['bottom'].set_position(('data', 0)) # x轴位置
ax.spines['left'].set_position(('data', 0)) # y轴位置

# 添加箭头 ,clip_on=False的作用是防止箭头被裁剪掉
ax.plot(1, 0, '>r',transform=ax.get_yaxis_transform(),clip_on=False) # transform=ax.get_yaxis_transform()表示箭头在y轴上
ax.plot(0, 1, '^b', transform=ax.get_xaxis_transform(),clip_on=False) # transform=ax.get_xaxis_transform()表示箭头在x轴上

# 点一个( /2,1)位置
ax.plot(0.5*np.pi, 1, 'og', markersize=8)
ax.plot(3.5*np.pi, -1, 'ob', markersize=8)

# 标注文字说明
ax.text(0.7*np.pi - 0.1, 1.2 - 0.1, "最高点($\pi$/2,1)", color='red', fontsize=10)
ax.text(3.7*np.pi - 0.1, -1.2 - 0.1, "最低点(7$\pi$/2,1)", color='red', fontsize=10)

plt.savefig('figures/section1/1.7.png') # 保存图形为文件
plt.show()
```

运行上述代码，将会显示一个包含不同样式箭头的图形窗口. 如下所示:

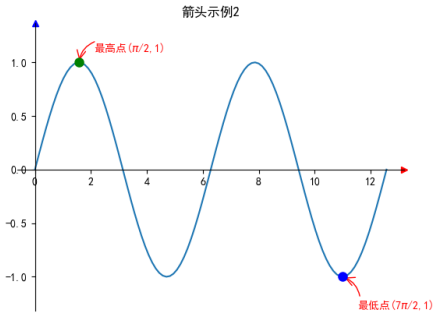

图 1.6: 不同样式箭头示例

7、 文字说明

在 matplotlib 中，可以通过多种方式来添加文字说明. 以下是一些常用的方法:

```python
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False


fig,ax = plt.subplots()
fig.suptitle("文字说明")
ax.set_title("latex文字说明")

ax.plot([0.5,1,1.5,2,2.5],[-0.5,-0.25,0,0.25,0.5],'og', markersize=2)
# 添加文字说明
ax.text(0.5, -0.5, "matplotlib可以插入latex公式", color='green', fontsize=12)
ax.text(1, -0.25, r"$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$", color='green', fontsize=15)
ax.text(1.5, 0, r"$\int_a^b f(x) \mathrm{d}x = F(b) - F(a)$", color='green', fontsize=15)
ax.text(2, 0.25, r"$e^{i \pi} + 1 = 0$", color='green', fontsize=15)
ax.text(2.5, 0.5, r"$\nabla \cdot \mathbf{E} = \frac{\rho}{\varepsilon_0}$", color='green', fontsize=15)



ax.set_xlim(0,3.5)
ax.set_ylim(-1, 1)

ax.grid(True)
plt.savefig('figures/section1/1.8.png') # 保存图形为文件
plt.show()
```

运行上述代码，将会显示一个包含文字说明的图形窗口. 如下所示:

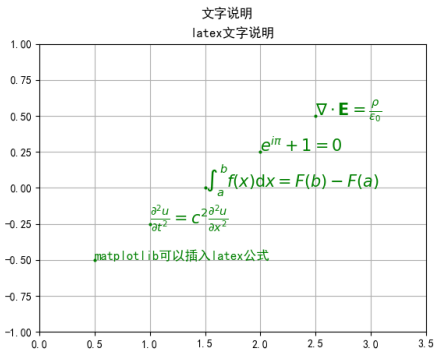

图 1.7: 文字说明示例

§2、 patches

在 matplotlib 中,patches 模块提供了许多用于绘制图形元素的类. 这些类可以用来创建各种形状, 如矩形、圆形、椭圆等. 以下是一些常用的 patches 类:

- `Rectangle`: 用于绘制矩形.
- `Circle`: 用于绘制圆形.
- `Ellipse`: 用于绘制椭圆.
- `Polygon`: 用于绘制多边形.

例如, 我们可以使用 patches.Rectangle 来创建一个正方形:

```python
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

fig, ax = plt.subplots()
# 创建一个正方形
rec = mpatches.Rectangle((0.2,0.2), 0.6, 0.6, facecolor='blue', edgecolor='black')
ax.add_patch(rec)

ax.plot(0.2,0.2,color = 'red', marker = 'o', markersize=10)

ax.set_xlim(0,1)
ax.set_ylim(0,1)
ax.set_title('正方形示例')
# x轴和y轴一样长
ax.set_aspect('equal')
# 保持图形
plt.savefig('figures/section2/2.1.png')
plt.show()
```

运行上述代码, 将会显示一个包含正方形的图形窗口. 如下所示:

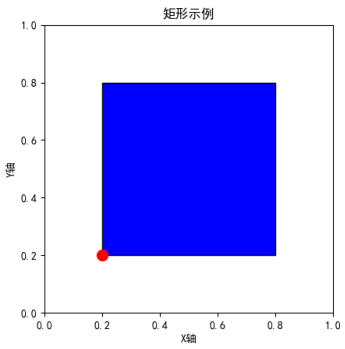

图 2.1: 正方形示例

1、 矩形

上面的例子已经展示了如何使用 *patches.Rectangle* 来创建一个矩形. 我们还可以通过设置不同的参数来调整矩形的属性, 如颜色、边框宽度等.

```python
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np

plt.rcParams["font.sans-serif"] = ['SimHei']
plt.rcParams["axes.unicode_minus"] = False
# 创建子图
fig,ax = plt.subplots(3,2,figsize=(10,15))

# angle=0表示不旋转,facecolor表示填充颜色,edgecolor表示边框颜色,linewidth表示边框宽度,alpha表示透明度
rect = mpatches.Rectangle((0.2,0.2),0.6,0.6,angle=0,facecolor="lightblue",edgecolor="blue",linewidth=2,alpha=0.7)
ax[0,0].add_patch(rect)
ax[0,0].set_xlim(0,1)
ax[0,0].set_ylim(0,1)
# adjustable参数设置为box时，长宽比固定
ax[0,0].set_aspect('equal', adjustable='box')

# angle=45表示旋转45度,默认是绕着左下角旋转
rect = mpatches.Rectangle((0.2,0.2),0.6,0.6,angle=45,facecolor="lightgreen",edgecolor="green",linewidth=2,alpha=0.7)
ax[0,1].add_patch(rect)
ax[0,1].set_xlim(0,1)
ax[0,1].set_ylim(0,1)
ax[0,1].set_aspect('equal', adjustable='box')

# 填充颜色和边框颜色,填充无颜色,边框填充红色
rect = mpatches.Rectangle((0.2,0.2),0.6,0.6,angle=0,facecolor='none',edgecolor="red",linewidth= 2,alpha=1)
ax[1,0].add_patch(rect)
ax[1,0].set_xlim(0,1)
ax[1,0].set_ylim(0,1)
ax[1,0].set_aspect('equal', adjustable='box')
# 修改长方形的位置在原点,默认是以左下角为原点
rect = mpatches.Rectangle((0,0),0.6,0.6,angle=0,facecolor="orange",edgecolor="brown",linewidth=2,alpha=0.7)
ax[1,1].add_patch(rect)
ax[1,1].set_xlim(0,1)
ax[1,1].set_ylim(0,1)
ax[1,1].set_aspect('equal', adjustable='box')
ax[1,1].plot(0,0,'ro',markersize = 10) # 标记原点位置

# 修改长方形的长和宽,宽为1,长为0.6
rect = mpatches.Rectangle((0.0,0.1),1,0.6,angle=0,facecolor="purple",edgecolor="none",linewidth=2,alpha=0.7)
ax[2,0].add_patch(rect)
ax[2,0].set_xlim(0,1)
ax[2,0].set_ylim(0,1)
ax[2,0].set_aspect('equal', adjustable='box')

# 修改长方形的透明度,alpha=0.3表示透明度为30%
rect1 = mpatches.Rectangle((0.2,0.2),0.6,0.6,angle=0,facecolor="red",edgecolor="none",linewidth=2,alpha=0.3)
rect2 = mpatches.Rectangle((0.3,0.3),0.6,0.6,angle=0,facecolor="blue",edgecolor="none",linewidth=2,alpha=0.4)
ax[2,1].add_patch(rect1)
ax[2,1].add_patch(rect2)
ax[2,1].set_xlim(0,1)
ax[2,1].set_ylim(0,1)
ax[2,1].set_aspect('equal', adjustable='box')

```

```
plt.savefig("figures/section2/2.2.png",dpi=300)
plt.show()
```

运行上述代码, 将会显示一个包含自定义矩形的图形窗口. 如下所示:

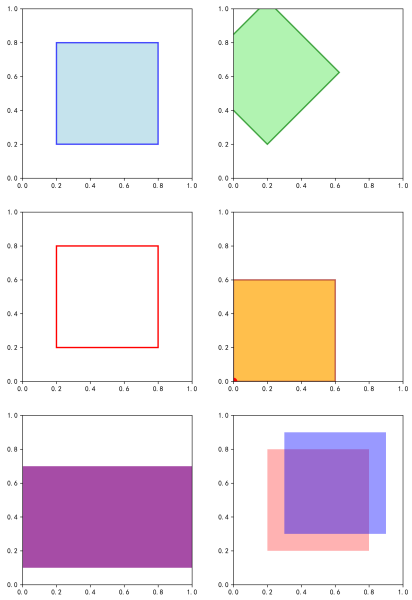

图 2.2: 自定义矩形示例

参考文献

[1] Matplotlib 官方文档, https://matplotlib.org/stable/contents.html

[2] NumPy 官方文档, https://numpy.org/doc/

Wu Jia Jun